Koneru Lakshmaiah Education Foundation

(Deemed to be University)

FRESHMAN ENGINEERING DEPARTMENT

A Project Based Lab Report

On

3X3 PUZZLE GAME

SUBMITTED BY:

I.D NUMBER	NAME
2300031072	T. LAKSHMI TEJASWI
2300031121	P.V. ROCHISHNA
2300031089	G. HRUDAY BHARADWAJ
2300031082	B. CHINA SUBBA REDDY

UNDER THE GUIDANCE OF

RAJESH KUMAR E

Assistant Professor

KL UNIVERSITY

Green fields, Vaddeswaram – 522 502 Guntur
Dt., AP, India.

DEPARTMENT OF BASIC ENGINEERING SCIENCES-1

CERTIFICATE

This is to certify that the project-based laboratory report entitled 3X3 PUZZLE GAME submitted by Mr./Ms**., T. LAKSHMI TEJASWI P.V.ROCHISHNA, G.HRUDAY BHARADWAJ, B. CHINA SUBBA REDDY** bearing Regd. No. **2300031072 2300031121, 2300031089, 2300031082,** to the **Department of Basic Engineering Sciences-1, KL University** in partial fulfilment of the requirements for the completion of a project based Laboratory in "**Computational Thinking for Structural Design** "course in I B Tech I Semester, is a Bonafide record of the work carried out by him under my supervision during the academic year 2023 – 2024.

PROJECT SUPERVISOR HEAD OF THE DEPARTMENT

Rajesh Kumar E Dr. D. Haritha

ACKNOWLEDGEMENTS

It is great pleasure for me to express my gratitude to our honorable President **Sri. Koneru Satyanarayana**, for giving the opportunity and platform with facilities in accomplishing the project-based laboratory report.

I express the sincere gratitude to our Director **Dr. A. Jagadeesh** for his administration towards our academic growth.

I express sincere gratitude to HOD-BES-1 **Dr. Haritha** for her leadership and constant motivation provided in successful completion of our academic semester. I record it as my privilege to deeply thank for providing us the efficient faculty and facilities to make our ideas into reality.

I express my sincere thanks to our project supervisor E. RAJESH KUMAR for his novel association of ideas, encouragement, appreciation and intellectual zeal which motivated us to venture this project successfully.

Finally, it is pleased to acknowledge the indebtedness to all those who devoted themselves directly or indirectly to make this project report success.

Project Associates…

ABSTRACT

3x3 puzzle game that combines strategy, logic, and spatial reasoning to engage players in a stimulating and entertaining experience. The game, titled "Puzzling Cubes," features a grid of nine cubes that players must manipulate to solve increasingly complex puzzles. The objective is to arrange the cubes in a specific order within the constraints of the 3x3 grid, testing the player's problemsolving skills.

"Puzzling Cubes" offers a diverse range of challenges, starting with fundamental puzzles to introduce players to the mechanics and gradually escalating in difficulty. The intuitive controls and minimalist design enhance accessibility while providing a visually appealing environment. The game's adaptive difficulty ensures that both casual players and puzzle enthusiasts find a suitable level of challenge.

The abstract concludes by highlighting the game's potential as an engaging and intellectually stimulating pastime, suitable for players of all ages. "Puzzling Cubes" aims to provide a satisfying and immersive puzzle-solving adventure, encouraging cognitive development and fostering a sense of accomplishment through its carefully crafted challenges.

INDEX

S.NO	TITLE	PAGE NO
1	Introduction	1
2	Aim of the Project	2
3	Software requirements	3
4	Code	4-8
5	Flow Chart	9-10
6	Algorithm	12
7	Implementation	13
8	Outputs screenshots	14-15

9 Conclusion 16

INTRODUCTION

3X3 PUZZLE GAME

Welcome, everyone!

Today, we're stepping into the realm of mind-bending challenges with the fascinating world of 3x3 puzzles. While the 3x3 grid may seem modest at first glance, it conceals a universe of complexity and strategic thinking.

A 3x3 puzzle is a deceptively simple game, consisting of nine tiles arranged in a 3x3 grid. The objective? Rearrange these tiles by sliding them into the empty space to form a complete picture or numerical sequence. The beauty lies in its simplicity, making it accessible to puzzle enthusiasts of all levels.

Don't let the limited grid fool you – solving a 3x3 puzzle demands more than meets the eye. It's a challenge that invites you to sharpen your wits, enhance your problem-solving skills, and indulge in the sheer joy of triumphing over seemingly straightforward puzzles.

But it's not just about fun and games. Engaging with 3x3 puzzles offers a range of cognitive benefits. From honing your analytical skills to improving memory and concentration, this seemingly simple game packs a powerful punch for your brain.

So, whether you're a casual gamer looking for a delightful pastime or a seasoned strategist ready to unravel complex patterns, the 3x3 puzzle game invites you to join an adventure where every slide and shift brings you closer to victory.

Get ready to immerse yourself in the enchanting world of 3x3 puzzles – where simplicity meets challenge, and every move takes you one step closer to unlocking the puzzle genius within. Let the adventure begin!

AIM

ADVANTAGES

- The code provides a practical application of fundamental C programming concepts and making educational resource for beginners
- It demonstrates the use of functions arrays, user input handling, and control flow in a simple understandable way
- The game provides straight forward and accessible introduction to programming logic and user input handling.
- The code is designed to portable across different operating systems allowing users to run it on windows, macOS and Linux

DISADVANTAGES

- The game is relatively simple and may lack the complexity and features found in more advanced puzzle games
- The user inter face is console based which may not provide the most engaging or visually appealing experience compared to graphical interfaces in modern games

- While the code checks for an invalid move during user input It could benefit from more robust error handling such as handling non integer inputs.

FUTURE ENHANCEMENTS

- Implementing graphical interface could enhance the user experience and make the game more visually appealing
  (GUI-GRAPHICAL USER INTERFACE)
- Introducing different difficulty levels or puzzle size could add complexity and interest to game
- Implementing score system or a timer could adda competitive element to the game
- Adding functionality to save the game.

SYSTEM REQUIREMENTS

- **SOFTWARE REQUIREMENTS**:

  The major software requirements of the project are as follows:

  Language : C

  Operating System: Windows, Linux, macOS.

- **HARDWARE REQUIREMENTS**:

The hardware requirements that map towards the software are as follows:

RAM : Minimum of 512MB

Processor : A processor with a clock speed of a few hundred megahertz or even less would likely be more than enough for running this code.

CODE

```
#include <stdio.h>

#include <stdlib.h>

#include <time.h>



void printPuzzle (int puzzle [3][3]) {

   for (int i = 0; i < 3; i++) {       for

(int j = 0; j < 3; j++) {                if

(puzzle[i][j]        ==        0)        {

printf("    ");                      }  else  {

printf("%d ", puzzle[i][j]);

          }

       }

       printf("\n");

   }

}
```

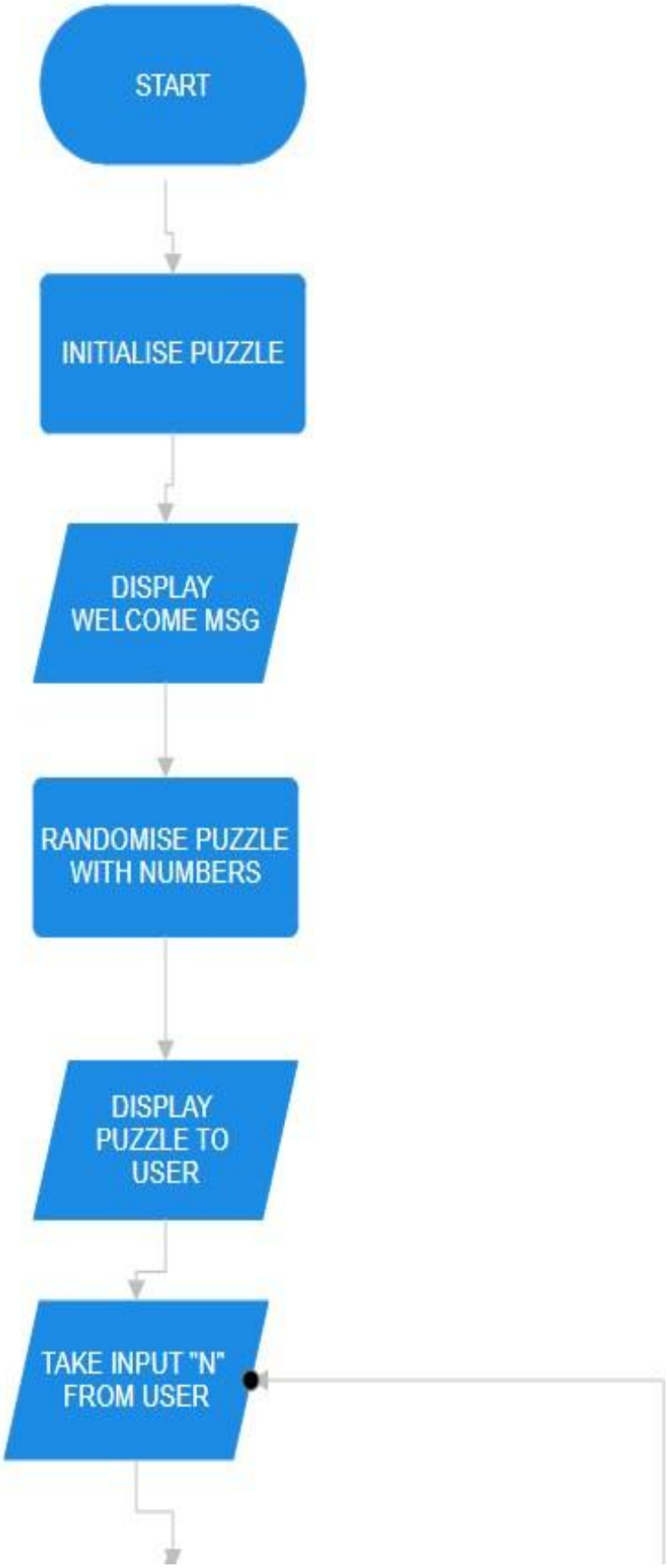
```
int isPuzzleSolved(int puzzle [3][3]) {
int count = 1;

   for (int i = 0; i < 3; i++) {
      for (int j = 0; j < 3; j++) {
if (puzzle[i][j] != count % 9) {
return 0;
         }
         count++;
      }
   }

   return 1;
}


void moveTile(int puzzle[3][3], int move) {
   int i, j;


   for (i = 0; i < 3; i++) {
for (j = 0; j < 3; j++) {
if (puzzle[i][j] == 0) {
break;
         }
      }
```

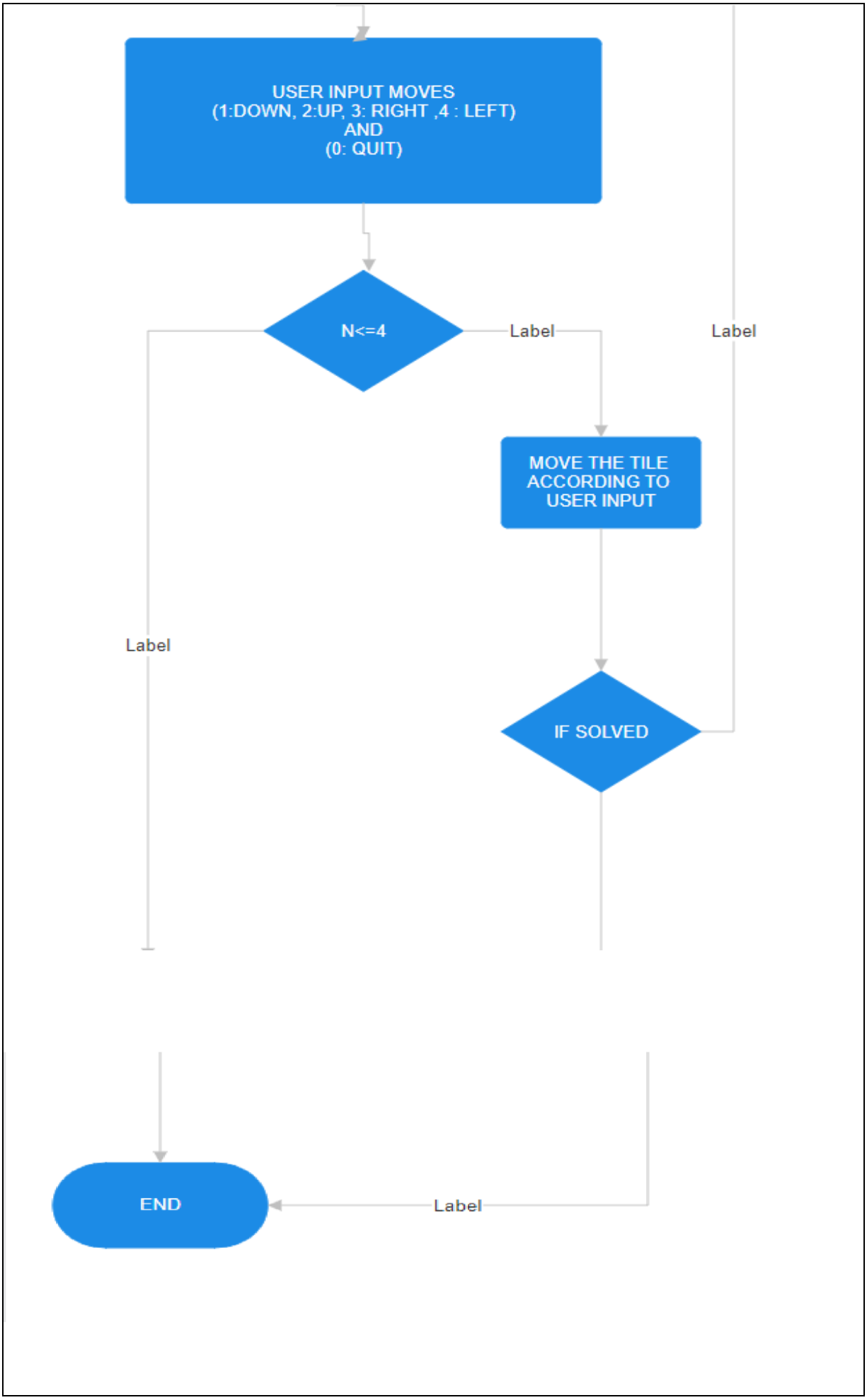
```
        if (puzzle[i][j] == 0) {
break;
        }
    }



    switch (move) {
        case 1:             if (i > 0) {
puzzle[i][j] = puzzle[i - 1][j];
puzzle[i - 1][j] = 0;
            }
            break;        case 2:             if (i
< 2) {                puzzle[i][j] = puzzle[i
+ 1][j];                puzzle[i + 1][j] = 0;
            }
            break;        case 3:             if (j
> 0) {                puzzle[i][j] =
puzzle[i][j - 1];                puzzle[i][j
- 1] = 0;
            }
            break;        case 4:             if (j
< 2) {                puzzle[i][j] =
puzzle[i][j + 1];                puzzle[i][j
+ 1] = 0;

            }
```

```
        break;
default:
        printf("Invalid move!\n");
    }
}

int main() {    int puzzle[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 0}}; // Initial
state of the puzzle    int move;    srand(time(NULL));



    for (int i = 0; i < 100; i++) {
move = rand() % 4 + 1;
moveTile(puzzle, move);
    }

    printf("Welcome to the 3x3 Puzzle Game!\n");

    do {
printPuzzle(puzzle);

        printf("Enter move (1: Down, 2: Up, 3: Right, 4: Left, 0: Quit): ");
scanf("%d", &move);

        if (move == 0) {
printf("Thanks for playing!\n");
        break;
        }
```

```
        moveTile(puzzle, move);

        if (isPuzzleSolved(puzzle)) {
printPuzzle(puzzle);            printf("Congratulations! You
solved the puzzle!\n");
            break;
        }

    } while (1);

    return 0;
}
```

FLOW CHART


START
INITIALISE PUZZLE
DISPLAY WELCOME MSG
RANDOMISE PUZZLE WITH NUMBERS
DISPLAY PUZZLE TO USER
TAKE INPUT "N" FROM USER


USER INPUT MOVES
(1:DOWN, 2:UP, 3: RIGHT ,4 : LEFT)
AND
(0: QUIT)
N<=4
Label
Label
MOVE THE TILE ACCORDING TO USER INPUT
Label
IF SOLVED
END
Label

ALGORITHM

STEP 1: Start.

STEP 2: Create a 3x3 grid with numbers from 1 to 8.

STEP 3: Shuffle the puzzle make the random numbers of valid moves to shuffle the puzzle.

STEP 4: Display the current state of the puzzle with 0 represented as a blank space

STEP 5: Cheak wheather the puzzle is solved or not.

STEP 6: if yes print a congratulatory message and exit.

STEP 7: if no then prompt the user that it is an invalid move

STEP 8: moving the tile by assigning the controls which is represented by numbers and their directions

STEP 9: Check the given input for movement of the number is valid or nor not

STEP 10: If yes update the puzzle and move the tile or move the number, and if no then give output as invalid input

STEP 11: In game, display welcome to the 3x3 puzzle game

STEP 12: Display the current state of the puzzle and accept the user given input and move according to the move

STEP 13: Repeat until the puzzle is solved, if the puzzle is solved then display congratulations you solved this puzzle

STEP 14: If not continue the game

STEP 15: END

IMPLEMENTATION

The topics that we have used in making this 3x3 puzzle game are

In conclusion, the provided C code for the 3x3 puzzle game serves as a valuable educational tool for novice programmers, offering a practical application of core C programming concepts. The modular structure of the code enhances readability and maintainability, demonstrating the effective use of functions and arrays. While the game's simplicity can be advantageous for educational purposes, its limited features and console-based interface may hinder its appeal for users seeking more sophisticated or visually engaging experiences.

Future enhancements, such as the implementation of a graphical user interface (GUI) and the introduction of different game levels, have the potential to address these limitations and broaden the game's appeal. Overall, the code presents a solid foundation for learning and can be further developed to incorporate additional features, making it a versatile starting point for those interested in expanding their programming skills and creating more intricate puzzle games.

OUTPUTS

Screen Shots

```
Welcome to the 3x3 Puzzle Game!
  1 2
8 3 5
6 7 4
Enter move (1: Down, 2: Up, 3: Right, 4: Left, 0: Quit): 4
1   2
8 3 5
6 7 4
Enter move (1: Down, 2: Up, 3: Right, 4: Left, 0: Quit): 4
1 2
8 3 5
6 7 4
Enter move (1: Down, 2: Up, 3: Right, 4: Left, 0: Quit): 2
1 2 5
8 3
6 7 4
Enter move (1: Down, 2: Up, 3: Right, 4: Left, 0: Quit): 3
1 2 5
8   3
6 7 4
Enter move (1: Down, 2: Up, 3: Right, 4: Left, 0: Quit): 1
1   5
8 2 3
6 7 4
Enter move (1: Down, 2: Up, 3: Right, 4: Left, 0: Quit): 2
1 2 5
8   3
6 7 4
Enter move (1: Down, 2: Up, 3: Right, 4: Left, 0: Quit): 2
1 2 5
```

```
Enter move (1: Down, 2: Up, 3: Right, 4: Left, 0: Quit): 1
1 2 5
8   3
6 7 4
Enter move (1: Down, 2: Up, 3: Right, 4: Left, 0: Quit): 2
1 2 5
8 7 3
6   4
Enter move (1: Down, 2: Up, 3: Right, 4: Left, 0: Quit): 3
1 2 5
8 7 3
  6 4
Enter move (1: Down, 2: Up, 3: Right, 4: Left, 0: Quit): 4
1 2 5
8 7 3
  6 4
Enter move (1: Down, 2: Up, 3: Right, 4: Left, 0: Quit): 3
1 2 5
8 7
3 6 4
Enter move (1: Down, 2: Up, 3: Right, 4: Left, 0: Quit): 2
1 2 5
8 7 4
3 6
Enter move (1: Down, 2: Up, 3: Right, 4: Left, 0: Quit): 0
Thanks for playing!

--------------------------------
Process exited after 61.88 seconds with return value 0
Press any key to continue . . .
```

CONCLUSION

In conclusion, the provided C code for the 3x3 puzzle game serves as a valuable educational tool for novice programmers, offering a practical application of core C programming concepts. The modular structure of the code enhances readability and maintainability, demonstrating the effective use of functions and arrays. While the game's simplicity can be advantageous for educational purposes, its limited features and console-based interface may hinder its appeal for users seeking more sophisticated or visually engaging experiences.

Future enhancements, such as the implementation of a graphical user interface (GUI) and the introduction of different game levels, have the potential to address these limitations and broaden the game's appeal. Overall, the code presents a solid foundation for learning and can be further developed to incorporate additional features, making it a versatile starting point for those interested in expanding their programming skills and creating more intricate puzzle games.